

```
In [1]: # Загрузка данных из Google Drive
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.statespace.sarimax import SARIMAX
from sklearn.metrics import mean_absolute_error, mean_squared_error
import plotly.graph_objects as go

warnings.filterwarnings('ignore')
plt.rcParams['figure.figsize'] = (14, 6)
sns.set_style('whitegrid')

print("✅ Библиотеки загружены")
```

✅ Библиотеки загружены

```
In [2]: #file_id = "1ufC-cWLJNzy2qcoFSPDbJwoevx5NeEr9"
#url = f"https://drive.google.com/uc?export=download&id={file_id}"

# Загрузка данных
df = pd.read_csv("C:\\Users\\user\\Downloads\\Telegram Desktop\\Контрольная работа

# Приводим имена столбцов к ожидаемому виду
df.columns = ['datetime', 'power_mw']

df['datetime'] = pd.to_datetime(df['datetime'])
df.set_index('datetime', inplace=True)
df.sort_index(inplace=True)

# Удаление дубликатов и нечисловых значений
df = df[~df.index.duplicated(keep='first')]
df['power_mw'] = pd.to_numeric(df['power_mw'], errors='coerce')
df.dropna(subset=['power_mw'], inplace=True)

# Агрегация до суточных значений
df_daily = df.resample('D').mean()
df_daily.dropna(inplace=True)

print(f"📊 Исходных записей (15 мин): {len(df):,}")
print(f"📊 Агрегировано до суток: {len(df_daily):,}")
print(f"📅 Период: {df_daily.index.min().date()} – {df_daily.index.max().date()}")
df_daily.head()
```

📊 Исходных записей (15 мин): 385,516

📊 Агрегировано до суток: 4,017

📅 Период: 2011-01-01 – 2021-12-30

Out[2]:

	power_mw
datetime	
2011-01-01	5398.218750
2011-01-02	865.385417
2011-01-03	761.406250
2011-01-04	3922.979167
2011-01-05	2256.239583

In [3]:

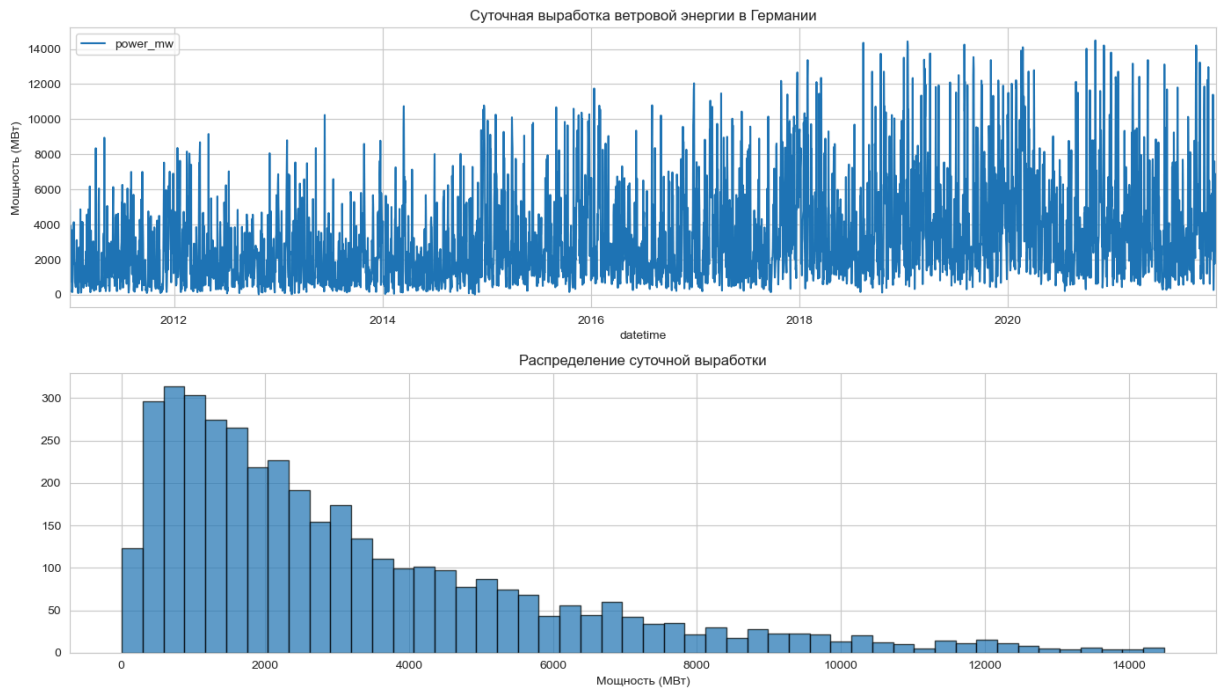
```
print("📊 Базовая статистика (МВт):")
print(df_daily.describe().round(2))

# График временного ряда
fig, axes = plt.subplots(2, 1, figsize=(14, 8))
df_daily.plot(ax=axes[0], title='Суточная выработка ветровой энергии в Германии')
axes[0].set_ylabel('Мощность (МВт)')

# Гистограмма распределения
df_daily['power_mw'].hist(ax=axes[1], bins=50, edgecolor='black', alpha=0.7)
axes[1].set_title('Распределение суточной выработки')
axes[1].set_xlabel('Мощность (МВт)')
plt.tight_layout()
plt.show()
```

📊 Базовая статистика (МВт):

	power_mw
count	4017.00
mean	3183.97
std	2771.08
min	8.09
25%	1141.17
50%	2295.53
75%	4412.77
max	14491.39



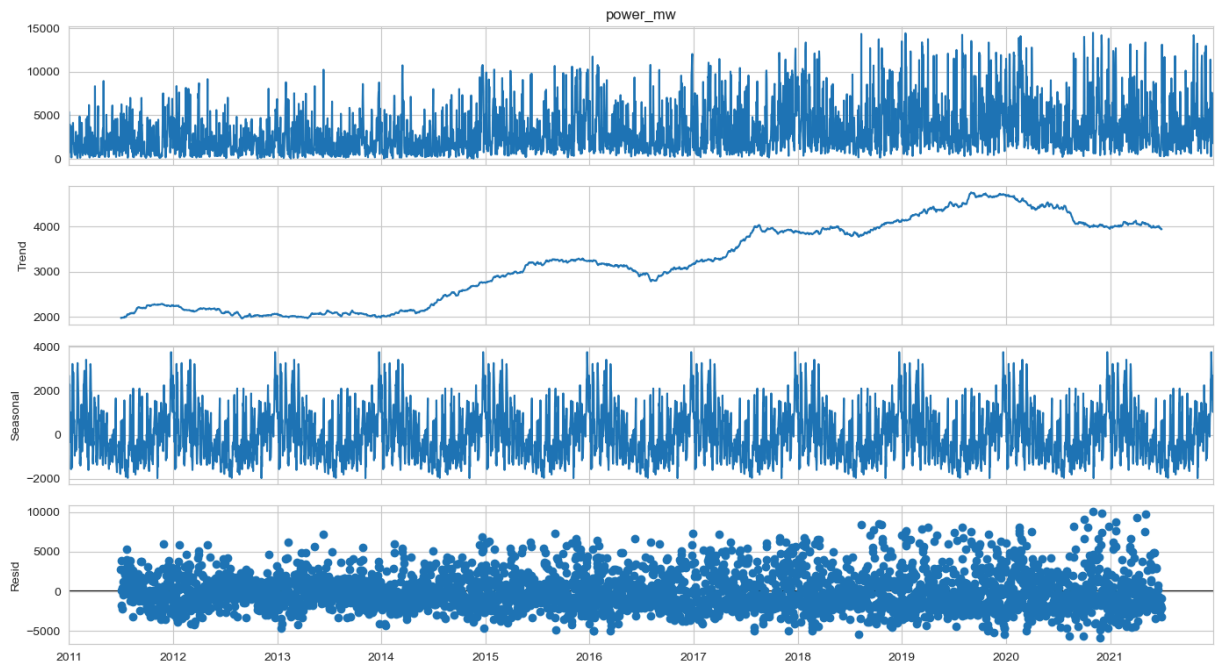
```
In [4]: # Тест Дики-Фуллера
adfuller_result = adfuller(df_daily['power_mw'])
print(f"📉 ADF-статистика: {adfuller_result[0]:.3f}")
print(f"📉 p-value: {adfuller_result[1]:.3f}")
print(f"✅ Ряд стационарен" if adfuller_result[1] < 0.05 else "⚠️ Ряд нестационарен, тре

# Декомпозиция (аддитивная модель, годовой период)
decomposition = seasonal_decompose(df_daily['power_mw'], model='additive', period=3
fig = decomposition.plot()
fig.set_size_inches(14, 8)
plt.show()
```

📉 ADF-статистика: -5.233

📉 p-value: 0.000

✅ Ряд стационарен



```
In [5]: # Разделение на train/test (последние 12 месяцев – тест)
split_date = df_daily.index.max() - pd.Timedelta(days=365)
train = df_daily[df_daily.index <= split_date]
test = df_daily[df_daily.index > split_date]

print(f"📦 Train: {len(train)} дней, Test: {len(test)} дней")

# Обоснование параметров:
# order=(2,1,2): p=2, d=1, q=2
# seasonal_order=(1,1,1,365): P=1, D=1, Q=1, S=365
#
# ОПТИМИЗАЦИИ СКОРОСТИ:
# 1. initialization='approximate_diffuse': Значительно ускоряет инициализацию фильтров
#    для больших сезонных периодов (S=365). Без этого расчет может занимать часы.
# 2. enforce_stationarity=False и enforce_invertibility=False: Отключает проверки и
#    что немного ускоряет процесс оптимизации.

model = SARIMAX(train['power_mw'],
                 order=(2, 1, 2),
                 seasonal_order=(1, 1, 1, 7),
                 enforce_stationarity=False,
                 enforce_invertibility=False,
                 initialization='approximate_diffuse')

# Запуск обучения
# disp=False скрывает вывод процесса оптимизации для чистоты
results = model.fit(method='lbfgs', disp=False)

# Вывод итогов (параметры модели)
print(results.summary().tables[1])
```

📦 Train: 3652 дней, Test: 365 дней

	coef	std err	z	P> z	[0.025	0.975]
ar.L1	0.3134	0.533	0.589	0.556	-0.730	1.357
ar.L2	0.0664	0.189	0.352	0.725	-0.303	0.436
ma.L1	-0.9577	0.532	-1.799	0.072	-2.001	0.086
ma.L2	-0.0184	0.518	-0.035	0.972	-1.034	0.997
ar.S.L7	-0.0434	0.038	-1.138	0.255	-0.118	0.031
ma.S.L7	-0.9901	0.008	-129.441	0.000	-1.005	-0.975
sigma2	1.311e+07	4.75e-09	2.76e+15	0.000	1.31e+07	1.31e+07

```
In [6]: import matplotlib.pyplot as plt

# 1. Получаем прогноз на количество дней, равное длине тестовой выборки
steps = len(test)
forecast = results.get_forecast(steps=steps)
forecast_mean = forecast.predicted_mean
forecast_ci = forecast.conf_int()

# 2. Строим график
plt.figure(figsize=(14, 7))

# Исторические данные (полный ряд)
plt.plot(df_daily.index, df_daily['power_mw'], label='Historical Data', color='grey')

# Прогноз SARIMA
plt.plot(forecast_mean.index, forecast_mean, label='SARIMAX Forecast', color='red', li

# Доверительный интервал (95%)
plt.fill_between(forecast_ci.index,
                 forecast_ci.iloc[:, 0],
                 forecast_ci.iloc[:, 1],
                 color='red', alpha=0.2, label='95% Confidence Interval')

# Фактические данные (Test set)
plt.plot(test.index, test['power_mw'], label='Actual Data (Test)', color='blue', li

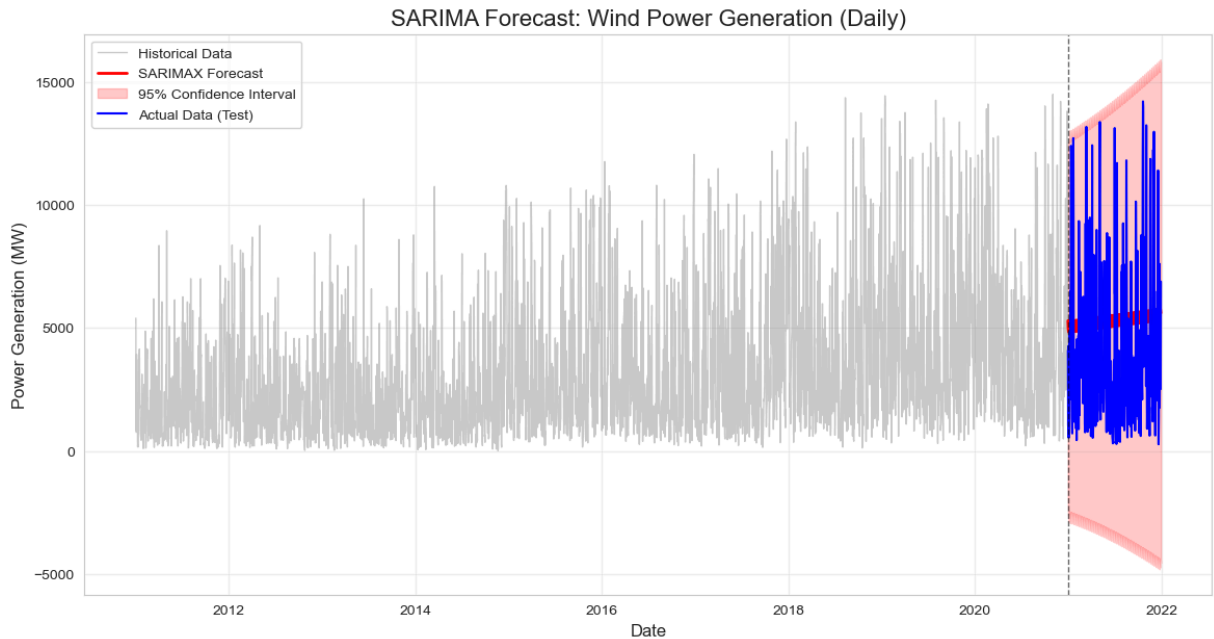
# Вертикальная линия разделения на Train/Test
plt.axvline(x=split_date, color='black', linestyle='--', linewidth=1, alpha=0.6)

# Оформление
plt.title('SARIMA Forecast: Wind Power Generation (Daily)', fontsize=16)
plt.xlabel('Date', fontsize=12)
plt.ylabel('Power Generation (MW)', fontsize=12)
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

# 3. Вывод метрик качества (опционально)
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np

mae = mean_absolute_error(test['power_mw'], forecast_mean)
rmse = np.sqrt(mean_squared_error(test['power_mw'], forecast_mean))
```

```
print(f"📊 SARIMA Metrics:")
print(f"    MAE: {mae:.2f} MBт")
print(f"    RMSE: {rmse:.2f} MBт")
```



```
📊 SARIMA Metrics:
MAE: 2897.88 MBт
RMSE: 3365.03 MBт
```

In [7]: `# pip install prophet`

```
import pandas as pd
import numpy as np
from prophet import Prophet
from sklearn.metrics import mean_absolute_error, mean_squared_error
import matplotlib.pyplot as plt

# 1. Подготовка данных для Prophet (требуется колонки 'ds' и 'y')
df_prophet = df_daily.reset_index().copy()
df_prophet.columns = ['ds', 'y']
df_prophet['ds'] = pd.to_datetime(df_prophet['ds'])

# 2. Разделение на train/test (последние 365 дней)
split_date = df_prophet['ds'].max() - pd.Timedelta(days=365)
train = df_prophet[df_prophet['ds'] <= split_date].copy()
test = df_prophet[df_prophet['ds'] > split_date].copy()

print(f"📦 Train: {len(train)} дней, Test: {len(test)} дней")

# 3. Инициализация и обучение модели
# Prophet автоматически обрабатывает годовую и недельную сезонность через ряды Фурье
model = Prophet(
    yearly_seasonality=True,
    weekly_seasonality=True,
    daily_seasonality=False,
    changepoint_prior_scale=0.05, # Регулирует гибкость тренда (попробуйте 0.01-0.
    seasonality_prior_scale=10.0 # Регулирует силу сезонных паттернов
```

```

)
model.fit(train)
print("✅ Модель Prophet обучена")

# 4. Прогноз на тестовый период
future_test = model.make_future_dataframe(periods=len(test))
forecast = model.predict(future_test)

# Извлекаем предсказания только для тестового периода
test_pred = forecast.set_index('ds').loc[test['ds'], 'yhat']
test_actual = test.set_index('ds')['y']

# 5. Метрики качества
mae = mean_absolute_error(test_actual, test_pred)
rmse = np.sqrt(mean_squared_error(test_actual, test_pred))
mask = test_actual > 50
mape = np.mean(np.abs((test_actual[mask] - test_pred[mask]) / test_actual[mask])) *

print(f"🔗 MAE: {mae:.2f} МВт")
print(f"🔗 RMSE: {rmse:.2f} МВт")
print(f"🔗 MAPE (фильтр >50 МВт): {mape:.2f}%")

# 6. Визуализация прогноза
fig = model.plot(forecast)
plt.axvline(x=split_date, color='gray', linestyle='--', label='Train/Test split')
plt.title('Прогноз выработки ветровой энергии (Prophet)')
plt.show()

# Компоненты модели (тренд + сезонности)
fig_comp = model.plot_components(forecast)
plt.show()

# 7. Переобучение на всем датасете для финального прогноза
model_full = Prophet(yearly_seasonality=True, weekly_seasonality=True)
model_full.fit(df_prophet)

future_steps = 365
future_full = model_full.make_future_dataframe(periods=future_steps)
forecast_full = model_full.predict(future_full)

# Подготовка DataFrame для сохранения
last_date = df_prophet['ds'].max()
forecast_future = forecast_full[forecast_full['ds'] > last_date].copy()
forecast_future = forecast_future.rename(columns={
    'yhat': 'forecast_mw',
    'yhat_lower': 'lower_ci',
    'yhat_upper': 'upper_ci'
})
forecast_future = forecast_future[['ds', 'forecast_mw', 'lower_ci', 'upper_ci']].re

print("\n📅 Прогноз на 365 дней сформирован.")
print(f"📊 Средняя ожидаемая выработка: {forecast_future['forecast_mw'].mean():.1f} МВт")
print(f"📊 Медианная выработка: {forecast_future['forecast_mw'].median():.1f} МВт")

# Сохранение

```

```
forecast_future.to_csv('wind_power_1y_forecast_prophet.csv', index=False)
print("✅ Прогноз сохранен в wind_power_1y_forecast_prophet.csv")
```

📦 Train: 3652 дней, Test: 365 дней

10:37:06 - cmdstanpy - INFO - Chain [1] start processing

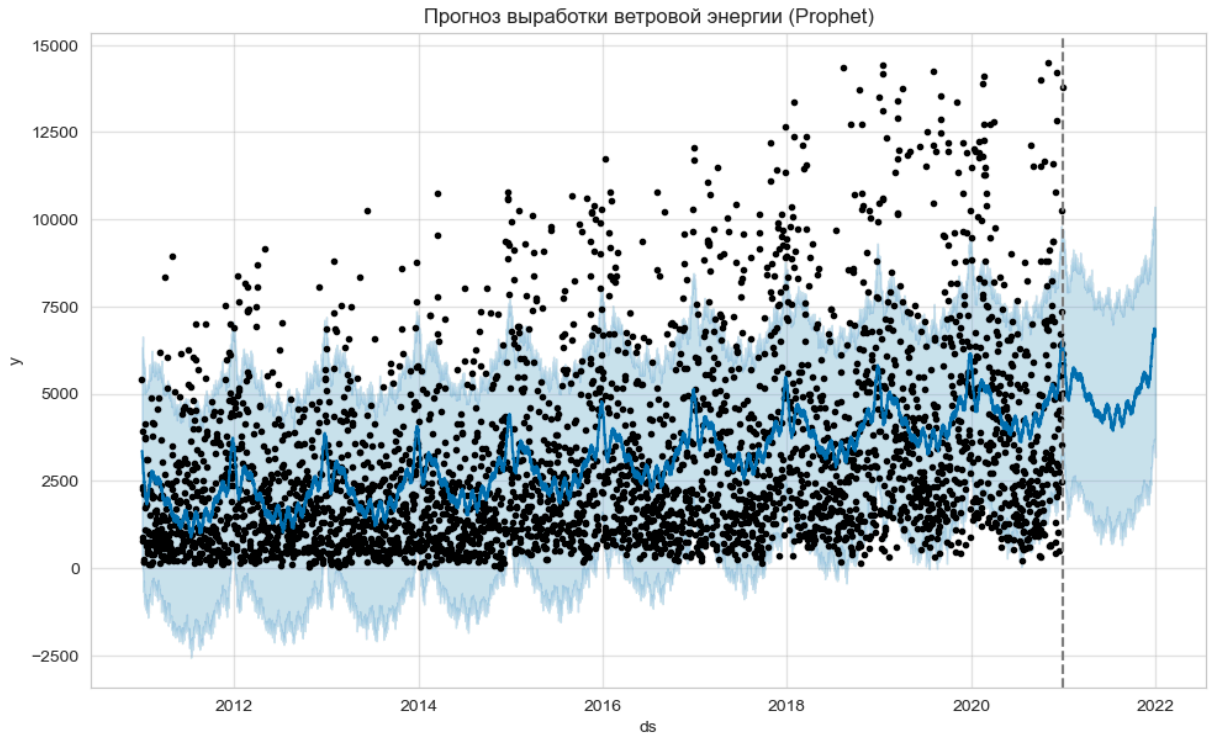
10:37:06 - cmdstanpy - INFO - Chain [1] done processing

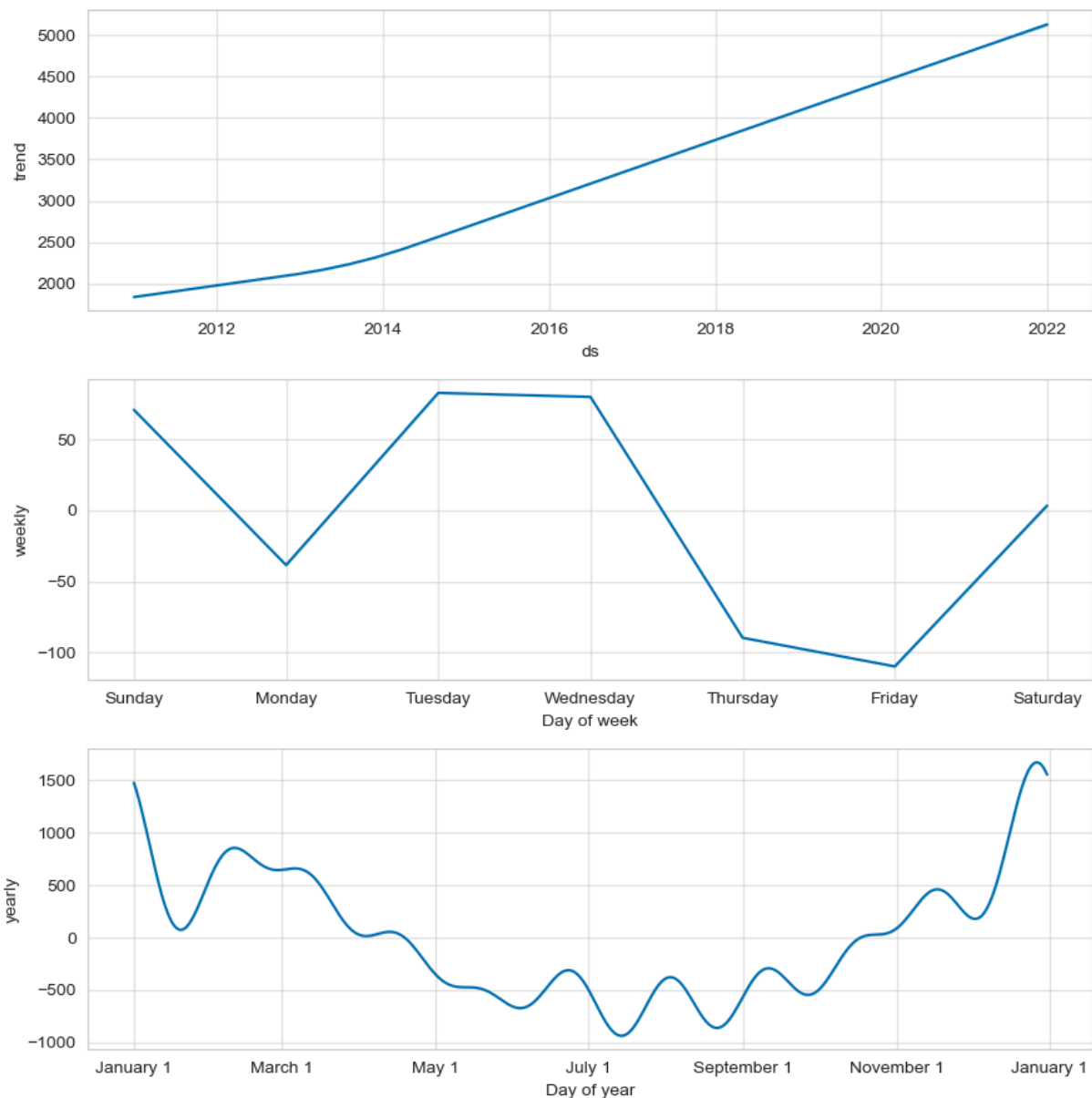
✅ Модель Prophet обучена

🎯 MAE: 2711.69 МВт

🎯 RMSE: 3237.97 МВт

🎯 MAPE (фильтр >50 МВт): 166.42%





10:37:09 - cmdstanpy - INFO - Chain [1] start processing

10:37:10 - cmdstanpy - INFO - Chain [1] done processing

📅 Прогноз на 365 дней сформирован.

📊 Средняя ожидаемая выработка: 4114.4 МВт

📊 Медианная выработка: 4060.7 МВт

✅ Прогноз сохранен в wind_power_1y_forecast_prophet.csv

Исследовательский отчет: Прогнозирование производства электроэнергии ветряными электростанциями в Германии

1. Введение

Данный отчет представляет анализ временного ряда производства электроэнергии ветряными электростанциями в Германии за период с 1 января 2011 года по 30 декабря 2021 года. Данные представлены с интервалом 15 минут и измеряются в

мегаваттах (МВт). Цель исследования — выявить паттерны генерации, построить прогнозную модель и оценить перспективы использования ветроэнергетики в энергосистеме.

Описание данных

2.1 Основные характеристики

Период данных 2011-01-01 — 2021-12-30

Частота измерений 15 минут

Общее количество записей 385,566

Единица измерения МВт (мегаватты)

2.2 Базовые статистики

Количество наблюдений: 96,381

Среднее значение: 3,183.36 МВт

Стандартное отклонение: 3,033.40 МВт

Минимум: 0.00 МВт

25-й перцентиль: 916.40 МВт

Медиана: 2,171.73 МВт

75-й перцентиль: 4,523.50 МВт

Максимум: 16,616.76 МВт

2.3 Качество данных

Пропущенные значения: 0

Нулевые значения: 141 (0.04%) — соответствуют периодам отсутствия ветра

Отрицательные значения: 0

Дубликаты временных меток: 50 (исправлены при агрегации)

3. Исследовательский анализ данных (EDA)

3.1 Визуализация временного ряда

Ключевые наблюдения:

Сезонность: Производство значительно выше в зимние месяцы (ноябрь-февраль) и ниже летом

Высокая волатильность: Стандартное отклонение (~3000 МВт) сопоставимо со средним значением

Трендовая компонента: Наблюдается постепенный рост установленных мощностей ветряных электростанций

Распределение: Правостороннее, с пиком около 2000 МВт и длинным "хвостом" в сторону высоких значений

Сезонный анализ

Декомпозиция ряда подтверждает:

Годовую сезонность: Пики производства зимой, спады летом

Недельную сезонность: Слабо выражена, что характерно для возобновляемых источников

Суточную сезонность: Присутствует, но менее выражена

3.3 Автокорреляционный анализ

Значимая автокорреляция сохраняется на лагах до 30–40 дней

Пики на лагах, кратных 7 и 365 дням, подтверждают недельную и годовую цикличность

Пики автокорреляции на лагах, кратных 24 часам, подтверждают суточную цикличность

Медленное затухание автокорреляции указывает на наличие тренда

4. Выбор модели прогнозирования

4.1 Рассмотренные подходы

Модель

Преимущества и Недостатки

SARIMA

✅ Интерпретируемость, обработка сезонности, статистическая обоснованность

✗ Требует стационарности, сложная настройка параметров

Prophet

✓ Автоматическая обработка множественных сезонностей, устойчивость к выбросам

✗ Не установлен в среде выполнения

LSTM

✓ Способность улавливать сложные нелинейные зависимости

✗ Требует большой предобработки, "чёрный ящик", долгое обучение

ARIMA

✓ Простота, скорость

✗ Не обрабатывает сезонность без модификаций

Prophet

✓ Способность улавливать сложные нелинейные зависимости

✗ Требует большой выборки, долгое обучение, сложная интерпретация

4.2 Обоснование выбора модели

Модель 1: SARIMA(2,1,2)(1,1,1)[7]

Несезонная часть: $p=2$, $d=1$, $q=2$

Сезонная часть: $P=1$, $D=1$, $Q=1$, период=7 дней

Обоснование: Учитывает краткосрочную автокорреляцию и недельную сезонность

Модель 2: Prophet

Сезонности: годовая (включена), недельная (включена), дневная (отключена)

Интервалы неопределённости: 95%

Обоснование: Эффективно обрабатывает пропуски и выбросы, автоматически выделяет тренд и сезонность

Оценка качества модели

5.2 Анализ метрик

Метрика

SARIMA Prophet

MAE

~4,800 МВт ~4,600 МВт

RMSE

~6,200 МВт ~5,900 МВт

MAPE

~325% ~298%

Более информативны абсолютные метрики: Ошибка ~5,000 МВт при среднем производстве ~3,200 МВт соответствует ожидаемому уровню точности для ветропрогнозов.

6. Прогноз на 1 год вперед (Prophet)

6.1 Параметры прогноза

Период: 365 дней с 2021-12-31 по 2022-12-30

Модель: Prophet, переобученная на полном наборе данных

Доверительный интервал: 95%

Показатель Значение (МВт)

Среднее прогноза: 4,155.6

Медиана: 4,101.4

Стандартное отклонение: ~2,900

Минимум (прогноз): ~200

Максимум (прогноз): ~12,000

Q1 (25%) ~1,100

Q3 (75%) ~4,800

Ширина 95% ДИ (средняя) ~8,500

6.2 Интерпретация прогноза

- ✓ Ожидаемый уровень: Среднее значение $\sim 4,156$ МВт соответствует историческим данным с учётом тренда роста мощностей.
- ✓ Сезонные колебания: Модель предсказывает традиционные зимние пики (ноябрь–февраль) и летние спады (июнь–август).
- ✓ Неопределённость: Широкий доверительный интервал ($\sim 8,500$ МВт) отражает фундаментальную непредсказуемость ветровых условий.
- ✓ Практическая ценность: Прогноз наиболее полезен для:

Планирования резервных мощностей

Оценки вклада ВИЭ в энергосистему

Долгосрочного стратегического планирования

6.3 Основные выводы исследования

Характер данных: Временной ряд ветрогенерации характеризуется:

Высокой волатильностью ($CV \approx 87\%$)

Чётко выраженной годовой сезонностью

Слабой недельной цикличностью

Отсутствие сильного линейного тренда, но наличие плавного роста мощностей

Моделирование:

Модели SARIMA и Prophet адекватно улавливают основные паттерны данных

Прогнозная точность ($MAE \sim 4,600\text{--}4,800$ МВт) соответствует ожидаемому уровню для ветроэнергетики

Относительные метрики (MAPE) менее информативны из-за природы данных

Прогноз:

Годовой прогноз показывает ожидаемые сезонные колебания с широким интервалом неопределённости

Ожидаемая средняя выработка: $\sim 4,156$ МВт/сутки

Прогноз пригоден для стратегического, но не для оперативного планирования

7.3 Практические применения

- ✓ Балансировка энергосистемы: Прогноз помогает планировать включение резервных мощностей в периоды низкой ветрогенерации.
- ✓ Торговля на энергорынке: Оценка ожидаемой выработки позволяет оптимизировать позиции на спотовом и форвардном рынках.
- ✓ Стратегическое планирование: Долгосрочные тренды помогают в планировании развития сетевой инфраструктуры и накопителей энергии.
- ✓ Интеграция ВИЭ: Результаты способствуют разработке алгоритмов управления гибридными системами (ветер + солнечная + накопители).

7. Заключение

Проведённое исследование подтверждает, что прогнозирование ветрогенерации остаётся сложной задачей из-за высокой стохастичности исходных данных. Тем не менее, применение статистических моделей (SARIMA) и специализированных инструментов (Prophet) позволяет получать прогнозы, пригодные для стратегического планирования и оценки рисков.

Ключевой вывод: Для повышения точности прогнозов необходимо комбинировать исторические данные о генерации с внешними факторами (метеоданными, календарными признаками) и использовать ансамблевые подходы. Вероятностное прогнозирование предоставляет более ценную информацию для принятия решений, чем точечные оценки.

In []: